LAB 2 · FIRMWARE DEVELOPMENT WITH ESP-IDF

Arduino Projects on ESP-IDF

Porting a sketch and its libraries.

Prepared by Pragyan Sen Deka, Electronics Engineer, Associate, IIM Calcutta Innovation Park Technology and Innovation Council



Figure 1. *The whole lab in one picture: nine steps that bring any Arduino project, libraries and all, into ESP-IDF.*

What	Used in this lab
Builds on	Lab 1 (installing ESP-IDF, building and flashing Hello world)
Example project	Scrolling text on a 4-in-1 MAX7219 LED matrix
Libraries	MD_Parola and MD_MAX72XX, both installed with the Arduino Library Manager
Board	ESP32-C61-DevKitC-1 (the method works for any ESP chip)
Software	ESP-IDF v5.5.5, Arduino core 3.3.12 (as in Lab 1)
Time needed	About 1–1.5 hours, most of it the first build

INSIDE THIS MANUAL

Contents

Click any line to jump to it. In Word you can also open **View → Navigation Pane** to see every heading in a sidebar.

START HERE **Before you begin**

CHAPTER 1 **How libraries work in each world**

CHAPTER 2 **Choosing safe pins**

CHAPTER 3 **The lab: your Parola project in 9 steps**

- Step 1: Find your libraries
- Step 2: Copy a working project
- Step 3: Copy the libraries into components
- Step 4: Give each library a CMakeLists.txt
- Step 5: Adapt the sketch
- Step 6: Tell main about the libraries
- Step 7: Set the target and build
- Step 8: Wire the display
- Step 9: Flash and watch

CHAPTER 4 **The general recipe for any library**

CHAPTER 5 **Troubleshooting**

CHAPTER 6 **Quick reference**

CHAPTER 7 **Glossary**

CHAPTER 8 **Check your understanding**

CHAPTER 9 **What's next**

START HERE

Before you begin

In Lab 1 you made the ESP32-C61 print **Hello world!**. That proved the tools work, but real projects use libraries: code other people wrote for displays, sensors and motors. In this lab you take a real Arduino project that uses two such libraries and make it run on the C61 through ESP-IDF.

The example is a sketch that scrolls text across a 4-in-1 LED matrix display. By the end, you will have a **recipe** you can follow for any Arduino project you already have.

What you will be able to do and explain

- Explain why ESP-IDF can't find Arduino libraries by itself, and how a `CMakeLists.txt` fixes that.
- Explain what header files are and what `REQUIRES` controls.
- Turn any Arduino library into an ESP-IDF component, whatever its folder layout.
- Choose pins that are safe on your board, and explain why some pins are off-limits.
- Recognise and fix the most common porting errors on your own.

What you need

Item	Details
Lab 1 finished	ESP-IDF v5.5.5 installed, and the working `hello_world` project in `D:\esp\projects`.
ESP32-C61 board + USB data cable	The same as in Lab 1.
4-in-1 MAX7219 LED matrix	The common "FC16" type: four 8×8 red LED blocks on one long board.
5 jumper wires	Female-to-female usually fits both the display and the board.
MD_Parola and MD_MAX72XX	Installed in the Arduino IDE with the Library Manager (Sketch → Include Library → Manage Libraries).

> NOTE
> This lab uses the same coloured boxes as Lab 1: **Why**, **Think of it like this**, **Common mistake**, **Try it**, **Checkpoint** and **New words**. Things you type are shown like `idf.py build`. Checkpoint answers are in Chapter 8.

CHAPTER 1

How libraries work in each world

1.1 The Arduino IDE fetches libraries for you. ESP-IDF doesn't.

When you write `#include <MD_Parola.h>` in the Arduino IDE, the IDE quietly searches your libraries folder, finds MD_Parola, compiles it and joins it to your sketch. You never see any of this.

ESP-IDF works differently. It builds only what it has been told about, and everything it builds must be a **component**: a folder with a `CMakeLists.txt` file inside. A library folder without that file is invisible to it.

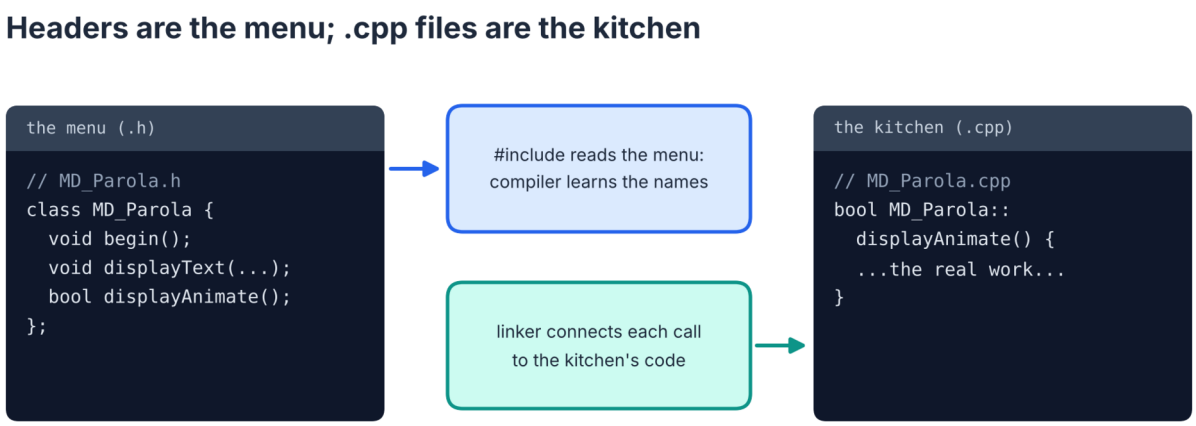


Figure 2. *The Arduino IDE searches for libraries itself. ESP-IDF only uses folders that carry a CMakeLists.txt, its "catalogue card".*

> THINK OF IT LIKE THIS
>
> The Arduino IDE is a librarian who walks the shelves and fetches any book you name. ESP-IDF is a stricter library: a book can only be borrowed if it has a **catalogue card**. In this lab you write the cards.

1.2 What a header file is

A library comes in two kinds of file. The **header** (`.h`) lists what the library offers: class names, function names, settings. The `.cpp` files contain the code that actually does the work.



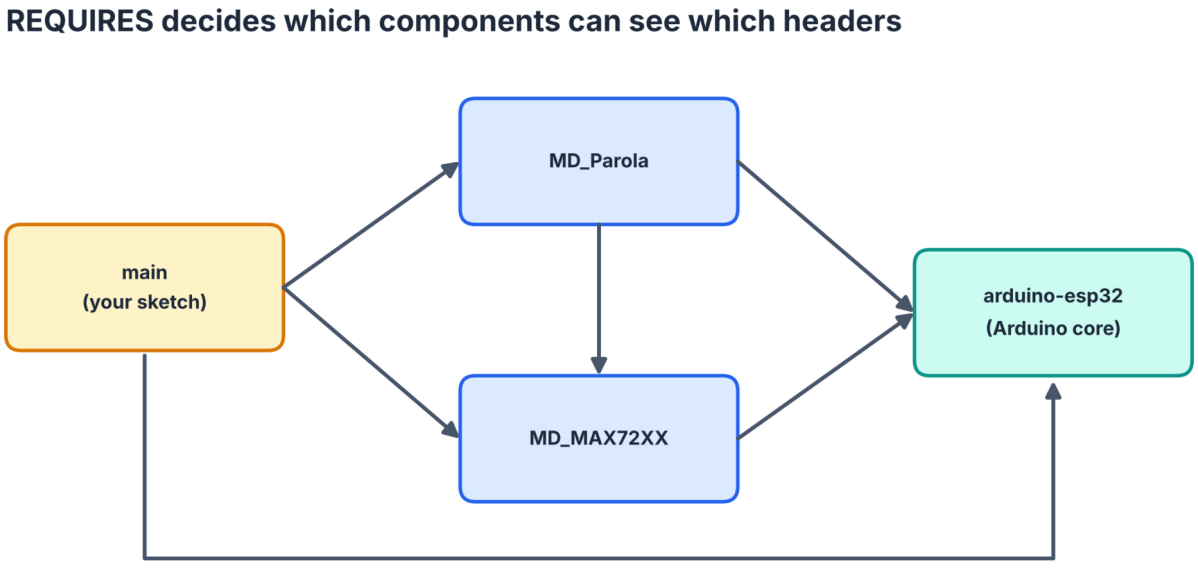
INCLUDE_DIRS in a CMakeLists.txt tells ESP-IDF where a component keeps its menu.

Figure 3. *#include reads the header so the compiler knows the names; the linker later connects each call to the real code.*

When the compiler meets `#include <MD_Parola.h>`, it must be able to **find** that header file. That is exactly what failed in this lab the first time (Step 6), so keep this picture in mind.

1.3 REQUIRES: who can see whom

Each component's `CMakeLists.txt` has a `REQUIRES` list: the other components it needs. A component can only `#include` headers from components on its own list. This is true for your own code too: the `main` folder is a component.



An arrow means "REQUIRES": the component at the start can #include the headers of the one at the end.

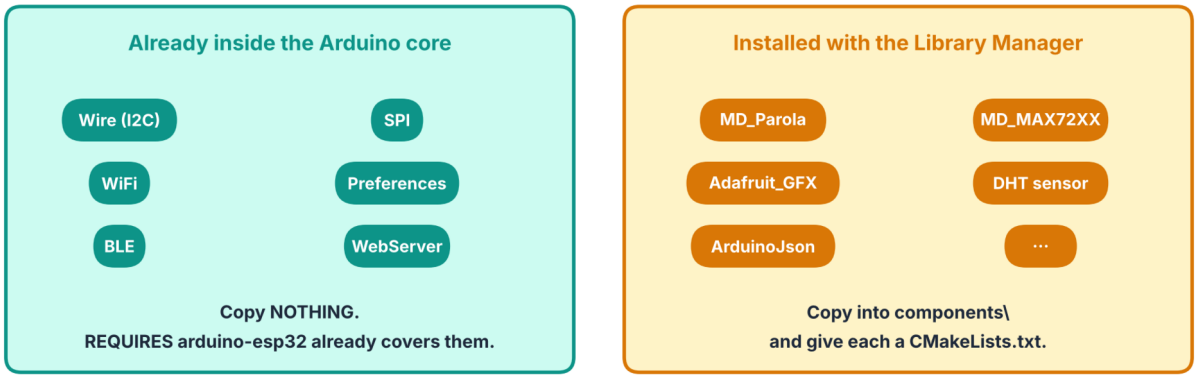
main had no arrows to the libraries at first, so the compiler said: MD_Parola.h: No such file or directory.

Figure 4. *The REQUIRES connections in the finished project. Every arrow had to be written into a CMakeLists.txt.*

1.4 Built-in libraries vs installed libraries

Not every library needs copying. Libraries such as `Wire`, `SPI` and `WiFi` are **part of the Arduino core** itself, which is already in your project as the `arduino-esp32` component. Only libraries you added with the Library Manager need to be brought in.

Built-in libraries vs libraries you installed



Careful: a folder called SPI or Wire in your Arduino libraries would clash with the core's own copy. Don't copy it.

Figure 5. *Built-in libraries come with the Arduino core; installed ones must be copied in and given a CMakeLists.txt.*

NEW WORDS IN THIS SECTION

Component: a folder ESP-IDF builds, recognised by the CMakeLists.txt inside it. Your main folder is one.

Header file (.h): a file that declares what a library offers, so other code can use its names.

REQUIRES: the list in a CMakeLists.txt of other components this one needs. It also decides whose headers it can see.

Library Manager: the Arduino IDE tool that downloads libraries into your Documents\Arduino\libraries folder.

CHECKPOINT

Your sketch uses `Wire.h` and `Adafruit_SSD1306.h`. Which of the two do you need to copy into the project?

CHAPTER 2

Choosing safe pins

The original sketch used GPIO 20, 21 and 19 for the display. On an ESP32-C61 board that would stop the board from working. This chapter explains why, and shows how to pick safe pins on any board.

2.1 Some pins already have a job

Your program is stored in the **flash memory** chip. The processor doesn't copy the whole program in at start-up; it keeps reading instructions from flash all the time it runs, through a set of wires called the **SPI bus**. On the C61, GPIO 20 is that bus's clock, and GPIO 19 and 21 are two of its data lines.

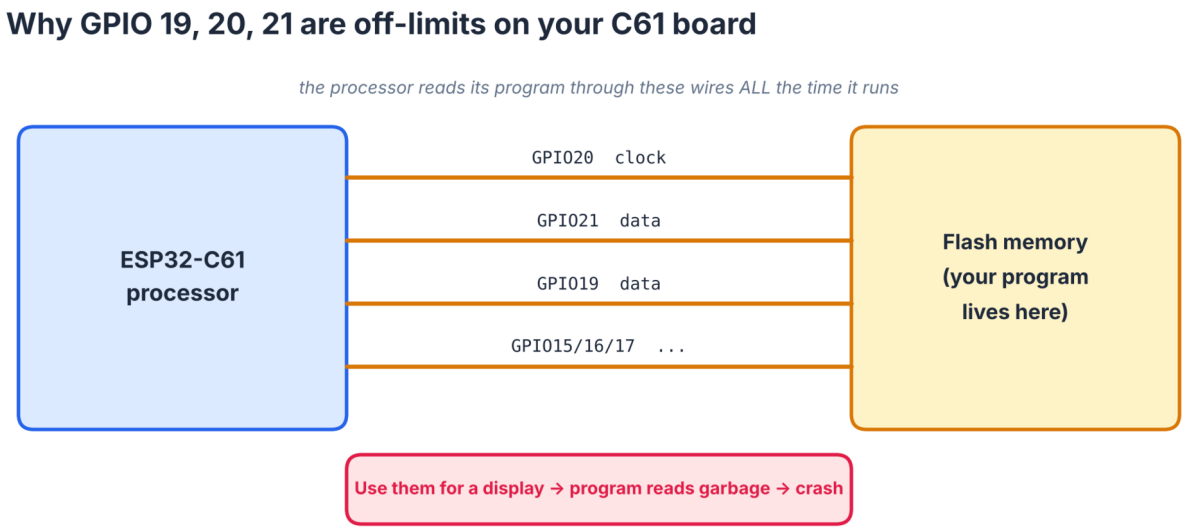


Figure 6. *If a sketch drives the flash wires, the processor reads garbage instead of its own program.*

> THINK OF IT LIKE THIS
>
> Imagine receiving instructions over a phone line while someone else shouts into the same line. The message gets corrupted. That is what happens if your display shares the flash chip's wires.

2.2 The pin map of your board

Espressif's user guide for the ESP32-C61-DevKitC-1 lists every header pin. GPIO 19–21 are not even brought out to the headers. Grouped by what they do:

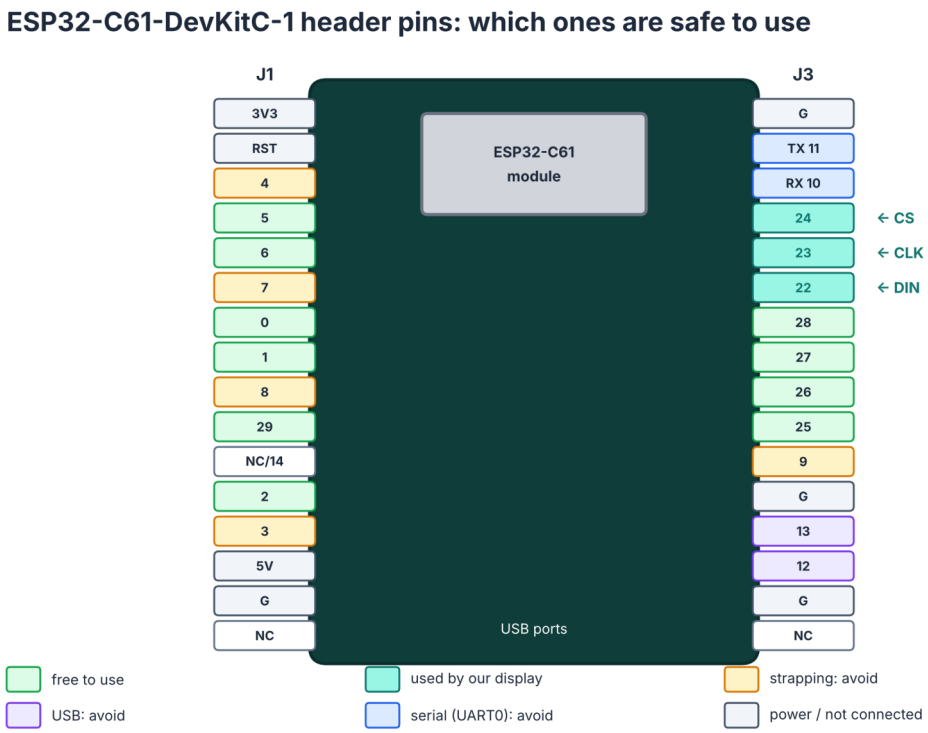


Figure 7. *Header pins of the ESP32-C61-DevKitC-1, coloured by whether they are safe. Check your own board's labels, as other boards differ.*

Pins	Their job	Use them?
GPIO 15–17, 19–21 (and 14)	Flash memory and PSRAM (SPI bus)	**Never**
GPIO 3, 4, 7, 8, 9	Strapping pins: read at start-up to choose a mode (e.g. BOOT = GPIO 9)	Avoid; GPIO 8 also drives the board's RGB LED
GPIO 12, 13	USB data lines	Avoid
GPIO 10, 11	Serial port (UART0) to the CP210x: your Serial.println()	Avoid
GPIO 0, 1, 2, 5, 6, 22–29	Free	**Yes**. We use 22, 23, 24

2.3 Picking pins on any other board

1. Find your board's **user guide** (search the board's name plus "user guide"). Look for its header pin table.
2. Avoid any pin the guide marks as **flash**, **PSRAM**, **SPI0/1**, **strapping**, **USB** or **UART0 TX/RX**.
3. Prefer pins that sit next to each other. It makes wiring tidier.

> NEW WORDS IN THIS SECTION
>
> **SPI bus:** a set of wires (clock, data, chip select) used to talk to chips such as flash memory and displays.
>
> **PSRAM:** extra RAM in a separate memory chip, also connected through SPI wires.
>
> **Strapping pin:** a pin the chip reads only at start-up to choose a mode (from Lab 1).

> **CHECKPOINT**
> A friend’s sketch uses GPIO 9 for a push button on the C61. What could go wrong when the board starts?

CHAPTER 3

The lab: your Parola project in 9 steps

Here is the sketch you are porting. It scrolls “hehehehe” across the display forever.

THE ORIGINAL ARDUINO IDE SKETCH

```
#include <MD_Parola.h>
#include <MD_MAX72xx.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 4
#define DATA_PIN 20
#define CLK_PIN 21
#define CS_PIN 19

MD_Parola P = MD_Parola(HARDWARE_TYPE, DATA_PIN, CLK_PIN, CS_PIN, MAX_DEVICES);

void setup() {
  P.begin();
  P.setIntensity(1);
  P.displayClear();
  P.displayText("hehehehe", PA_RIGHT, 50, 0, PA_SCROLL_RIGHT, PA_SCROLL_RIGHT);
}

void loop() {
  if (P.displayAnimate()) {
    P.displayReset();
  }
}
```

The two `#include` lines at the top tell you which libraries it needs: **MD_Parola** (the text effects) and **MD_MAX72XX** (which drives the LED chips). MD_Parola is built on top of MD_MAX72XX, so you need both.

STEP 1

Find your libraries

Goal: confirm both libraries are on your computer, and learn where the Arduino IDE keeps them.

WHY ARE WE DOING THIS?

The Arduino IDE downloaded both libraries into one folder. ESP-IDF doesn't know that folder exists, so first you need to know where the libraries are.

TYPE THIS

```
dir $HOME\Documents\Arduino\libraries
```

`$HOME` means your Windows user folder, such as `C:\Users\pragy`. On many computers the Documents folder is inside OneDrive, so the real path may be `C:\Users\pragy\OneDrive\Documents\Arduino\libraries`. Use whichever one lists your libraries.

YOU SHOULD SEE (similar to this)

```
    Directory: C:\Users\pragy\OneDrive\Documents\Arduino\libraries
...
dar--l    MD_MAX72XX
dar--l    MD_Parola
...
```

TRY IT

Every Arduino library has a `library.properties` file describing it. Run `type <your libraries folder>\MD_Parola\library.properties` and find the line `depends=MD_MAX72XX`. That line is how you discover which other libraries a library needs.

STEP 2

Copy a working project

Goal: make a new project folder for this sketch by copying the recipe files of hello_world.

WHY ARE WE DOING THIS?

`hello_world` already builds on your C61 and already has the Arduino core version pinned. Copying it saves repeating Steps 2–4 of Lab 1. But you copy only the **recipe**, not the files ESP-IDF made automatically.

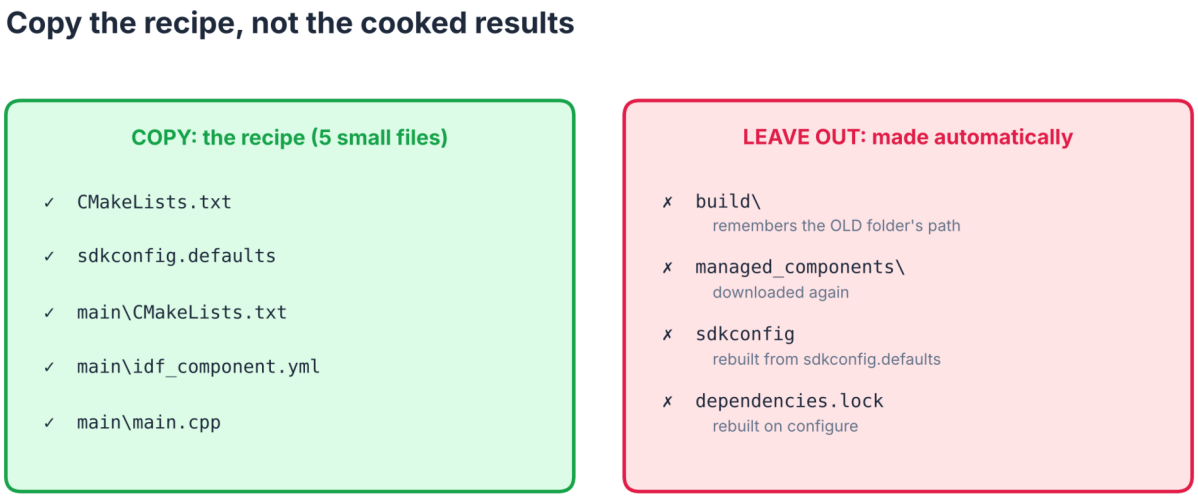


Figure 8. *Copy the five recipe files; leave out everything ESP-IDF creates by itself.*

TYPE THIS

```
cd D:\esp\projects
mkdir parola_display\main
copy hello_world\CMakeLists.txt parola_display\
copy hello_world\sdkconfig.defaults parola_display\
copy hello_world\main\* parola_display\main\
```

`mkdir parola_display\main` creates the project folder and its `main` folder in one go. The `*` means “every file in that folder”.

Why no drive letter is needed

After `cd D:\esp\projects`, the terminal is “in” that folder. A path without a drive letter, a **relative path**, is looked up starting from there. A path with the drive letter is an **absolute path** and works from anywhere.



Figure 9. *Current folder + relative path = the full, absolute path.*

CHECK

```
dir parola_display
dir parola_display\main
```

You should see `CMakeLists.txt` and `sdkconfig.defaults` plus a `main` folder, and three files inside `main`. `idf_component.yml` should be the small file (about 63 bytes) you pinned in Lab 1.

> **COMMON MISTAKE**
>
> The `build` folder remembers the **full path** of the project it was made in. Copying it into a new project makes the build look in the wrong place and fail with confusing errors. Never copy `build\`.

STEP 3

Copy the libraries into components

Goal: put both libraries inside the project, where ESP-IDF will look for them.

> **WHY ARE WE DOING THIS?**
>
> ESP-IDF automatically looks inside a project for a folder named exactly **components**. Every folder in it can become a component of the project.

TYPE THIS

```
cd parola_display
mkdir components
Copy-Item -Recurse C:\Users\pragy\OneDrive\Documents\Arduino\libraries\MD_MAX72XX
components\
Copy-Item -Recurse C:\Users\pragy\OneDrive\Documents\Arduino\libraries\MD_Parola
components\
```

Use your own libraries path from Step 1. `copy` handles only single files, so for whole folders you use PowerShell's `Copy-Item`. `-Recurse` means "also copy everything inside, including sub-folders". If the libraries are in OneDrive, Windows may take a few seconds to download them first.

CHECK

```
dir components\MD_Parola
```

YOU SHOULD SEE (similar to this)

```
d-----    docs
d-----    examples
d-----    hardware
d-----    media
d-----    src
-a----    library.properties
-a----    README.md
...
```

The code lives in **src**. Everything else (docs, examples, media) is there for people and for the Arduino IDE. That `src` folder decides what goes on the catalogue card in Step 4.

STEP 4

Give each library a CMakeLists.txt

Goal: write a catalogue card for each library so ESP-IDF treats it as a component.

> **WHY ARE WE DOING THIS?**
> Without a `CMakeLists.txt`, ESP-IDF ignores the library folders, and the build would fail with "MD_Parola.h: No such file or directory". You start with MD_MAX72XX because MD_Parola is built on top of it.

MD_MAX72XX first

TYPE THIS

```
dir components\MD_MAX72XX
notepad components\MD_MAX72XX\CMakeLists.txt
```

Check that `src` appears in the list. Notepad will ask whether to create a new file: click **Yes**, paste these lines, save and close.

components\MD_MAX72XX\CMakeLists.txt

```
idf_component_register(SRC_DIRS "src"
                       INCLUDE_DIRS "src"
                       REQUIRES arduino-esp32)
```

Then MD_Parola

TYPE THIS

```
notepad components\MD_Parola\CMakeLists.txt
```

components\MD_Parola\CMakeLists.txt

```
idf_component_register(SRC_DIRS "src"
                       INCLUDE_DIRS "src"
                       REQUIRES arduino-esp32 MD_MAX72XX)
```

The only difference is `MD_MAX72XX` added to `REQUIRES`. Inside its own code, MD_Parola does `#include <MD_MAX72xx.h>`, so its card must say it needs MD_MAX72XX. Otherwise it couldn't see that header.

Reading the card

Reading a component's CMakeLists.txt

components\MD_Parola\CMakeLists.txt

```
idf_component_register(
    SRC_DIRS     "src"
    INCLUDE_DIRS "src"
    REQUIRES     arduino-esp32 MD_MAX72XX
)
```

ESP-IDF function: register a component
compile every .c/.cpp file in src
other code finds my .h files in src
components I need (folder names, exact)

Rules: KEYWORDS in capitals · values separated by spaces or new lines · quotes optional · # starts a comment

Line breaks and lined-up spaces are only for readability: on one line it means exactly the same.

Figure 10. *Each part of a component's CMakeLists.txt, and the rules the CMake language follows.*

> **COMMON MISTAKE**
>
> The name after `REQUIRES` must match the **folder name** exactly, capitals included: `MD_MAX72XX`, not `MD_MAX72xx` (which is how the header file is spelled). ESP-IDF names each component after its folder.

CHECK

```
type components\MD_MAX72XX\CMakeLists.txt
type components\MD_Parola\CMakeLists.txt
```

STEP 5

Adapt the sketch

Goal: put the sketch into main.cpp with the two changes ESP-IDF needs.

TYPE THIS

```
notepad main\main.cpp
```

Delete everything in it (it still holds hello_world), paste this, save and close:

main\main.cpp

```
#include "Arduino.h"      // CHANGE 1: ESP-IDF doesn't add this for you
#include <MD_Parola.h>
#include <MD_MAX72xx.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW

#define MAX_DEVICES 4
#define DATA_PIN 22       // CHANGE 2: safe pins on the C61 board
#define CLK_PIN  23
#define CS_PIN   24

MD_Parola P = MD_Parola(HARDWARE_TYPE, DATA_PIN, CLK_PIN, CS_PIN, MAX_DEVICES);

void setup() {
  P.begin();
  P.setIntensity(1);
  P.displayClear();
  P.displayText("hehehehe", PA_RIGHT, 50, 0, PA_SCROLL_RIGHT, PA_SCROLL_RIGHT);
  // displayText(text, align, speed, pause, effectIn, effectOut)
}

void loop() {
  if (P.displayAnimate()) {   // true = animation finished
    P.displayReset();         // rewind and play it again
  }
}
```

Only two changes to the sketch

Arduino IDE version

```
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
...
#define DATA_PIN 20
#define CLK_PIN  21
#define CS_PIN   19
```

ESP-IDF version (main.cpp)

```
#include "Arduino.h"
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
...
#define DATA_PIN 22
#define CLK_PIN  23
#define CS_PIN   24
```

① ESP-IDF doesn't add Arduino.h for you ② GPIO 19–21 belong to the flash chip on the C61

Figure 11. *Everything else in the sketch stays exactly as it was.*

- **`#include "Arduino.h"`**: the Arduino IDE adds this line to every sketch without telling you. ESP-IDF doesn't, so without it names like `setup`, `loop` and `delay` would be unknown.
- **Pins 22, 23, 24**: GPIO 19–21 belong to the flash chip on the C61 (Chapter 2).

How the sketch works

Because you pass three pin numbers, MD_MAX72XX doesn't use the chip's SPI hardware. It switches the three pins on and off in software, called **bit-banging**, so any three free pins work. `HARDWARE_TYPE FC16_HW` tells the library how the LED chips are wired on this kind of module; the wrong type gives mirrored or scrambled text.

How the sketch keeps the text scrolling

loop() runs thousands of times a second; Parola only moves the text when 50 ms have passed.

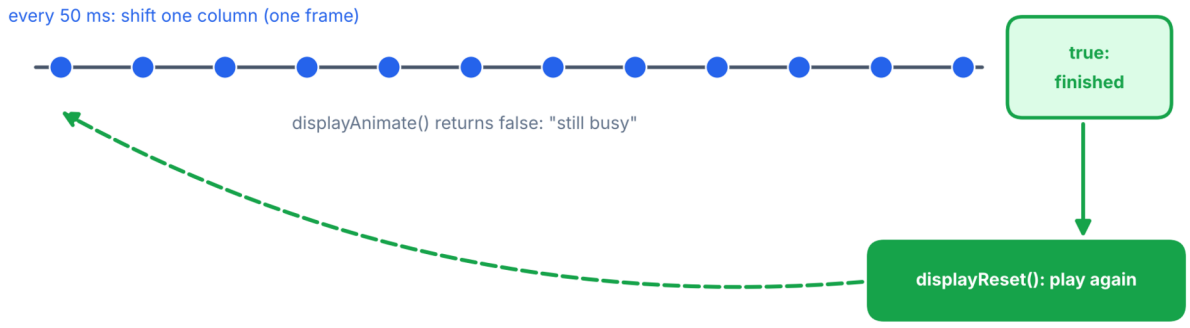


Figure 12. *loop() calls displayAnimate() over and over; Parola moves the text one column every 50 ms and reports when it has finished.*

STEP 6

Tell main about the libraries

Goal: let your own code see the two libraries by adding them to main's REQUIRES.

WHY ARE WE DOING THIS?

`main` is a component too, and it can only see headers from components on its own `REQUIRES` list. The `main\CMakeLists.txt` you copied from hello_world mentions only the Arduino core. This step is the one most people forget.

TYPE THIS

```
notepad main\CMakeLists.txt
```

main\CMakeLists.txt

```
idf_component_register(SRCS "main.cpp"
                       INCLUDE_DIRS "."
                       REQUIRES arduino-esp32 MD_Parola MD_MAX72XX)
```

`SRCS "main.cpp"` lists your one code file by name. `INCLUDE_DIRS "."` means "this folder". The new `REQUIRES` lets `main` see both libraries, because your sketch includes both headers.

The error you get if main's REQUIRES is missing the libraries

```
main.cpp:2:10: fatal error: MD_Parola.h: No such file or directory
    2 | #include <MD_Parola.h>
      |          ^~~~~~~~~~~~~
```

Before (copied from hello_world)

```
REQUIRES arduino-esp32
→ main can't see MD_Parola.h
```

After

```
REQUIRES arduino-esp32
         MD_Parola MD_MAX72XX
```

Figure 13. *What you see if you skip this step, and the one-line fix.*

NOTE

If the build fails, ESP-IDF may also print `BUG: component_requirements.py: cannot match original component filename`. That comes from a helper that tries to suggest a fix and stumbled itself. Ignore it and read the first `error:` line above it.

STEP 7

Set the target and build

Goal: configure the new project for the C61 and compile everything.

TYPE THIS

```
idf.py set-target esp32c61
```

As in Lab 1, this picks the C61's compiler, creates `sdkconfig` and downloads the Arduino core. This time it also scans `components` and finds MD_MAX72XX and MD_Parola through their new cards. A mistake in a card shows up here as a red error.

YOU SHOULD SEE (similar to this)

```
-- Configuring done (99.7s)
-- Generating done (2.2s)
-- Build files have been written to: D:/esp/projects/parola_display/build
```

TYPE THIS

```
idf.py build
```

This is the first real test of the libraries: the compiler builds MD_MAX72XX, MD_Parola and your `main.cpp` for the C61, and the linker joins them. It is a new project, so the whole Arduino core is compiled again: allow 5–15 minutes. Yellow warnings are fine.

YOU SHOULD SEE (similar to this)

```
Project build complete. To flash, run:
 idf.py flash
...
```

The finished project folder

parola_display	the project	
├─ CMakeLists.txt	copied from hello_world	
├─ sdkconfig.defaults	copied from hello_world	
├─ **main**	YOUR component	
│ ├─ CMakeLists.txt	REQUIRES now lists both libraries	you edited
│ ├─ idf_component.yml	Arduino core pinned to ^3.3.12	
│ └─ main.cpp	your sketch + #include "Arduino.h"	you edited
└─ **components**	ESP-IDF looks here automatically	you created
├─ MD_MAX72XX\	copied from Arduino libraries	
│ ├─ CMakeLists.txt	its catalogue card	you created
│ └─ src\	the library's code	
└─ MD_Parola\	copied from Arduino libraries	
├─ CMakeLists.txt	its card: also REQUIRES MD_MAX72XX	you created
└─ src\	the library's code	

Figure 14. *The finished project. Everything marked was created or edited by you in Steps 2–6.*

STEP
8

Wire the display

Goal: connect the LED matrix to the safe pins, with the board unplugged.

WHY ARE WE DOING THIS?

Connecting wires while the board is powered can briefly short pins together. Unplug first: it is a good habit with every board.

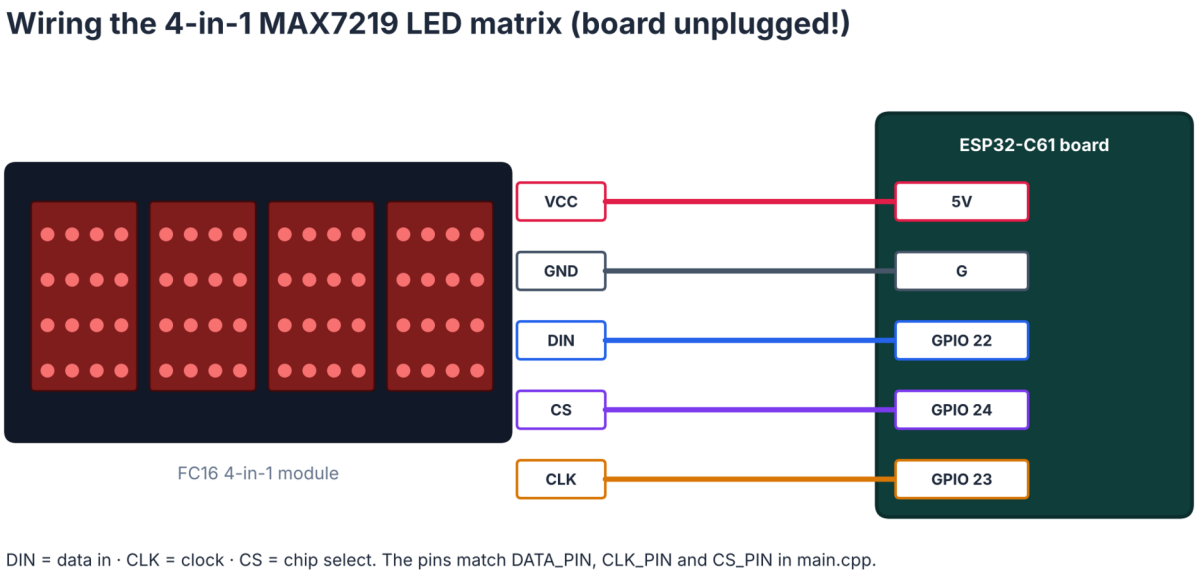


Figure 15. *Five wires: power, ground and the three signal pins from main.cpp.*

Display pin	Board pin	Why
VCC	5V (left header J1)	Power for the LEDs
GND	G (any ground)	Common ground
DIN	22 (right header J3)	DATA_PIN 22
CLK	23 (J3)	CLK_PIN 23
CS	24 (J3)	CS_PIN 24

NOTE

The display runs on 5 V, while the C61's pins give 3.3 V signals. Strictly, a MAX7219 at 5 V wants a little more than 3.3 V for a "high" signal, but most of these modules work fine with it in practice. If the display stays dark or shows random dots, this is the first thing to suspect: try powering the module from 3V3 instead, or use a level shifter.

STEP 9

Flash and watch

Goal: write the program to the board and see the text scroll.

Plug the board in and use your COM port from Lab 1 (check Device Manager if you changed USB socket):

TYPE THIS

```
idf.py -p COM19 flash monitor
```

You will see the familiar esptool messages ending in `Hash of data verified` and `Hard resetting`. Then the board restarts and the text starts scrolling. This sketch doesn't print anything over serial, so the monitor shows only the boot messages and then stays quiet: the display is where the result appears. Press **Ctrl +]** to leave the monitor.

> YOU DID IT!
> "hehehehe" scrolling across the matrix means an existing Arduino project, with two Library Manager libraries, is running on a board the Arduino IDE can't program. You now have a recipe that works for your other projects too.

> TRY IT
> Change the text to your name, the speed from `50` to `25`, and the effect to `PA_SCROLL_LEFT`. Run `idf.py -p COM19 flash monitor` again. The rebuild should take seconds, not minutes. Why?

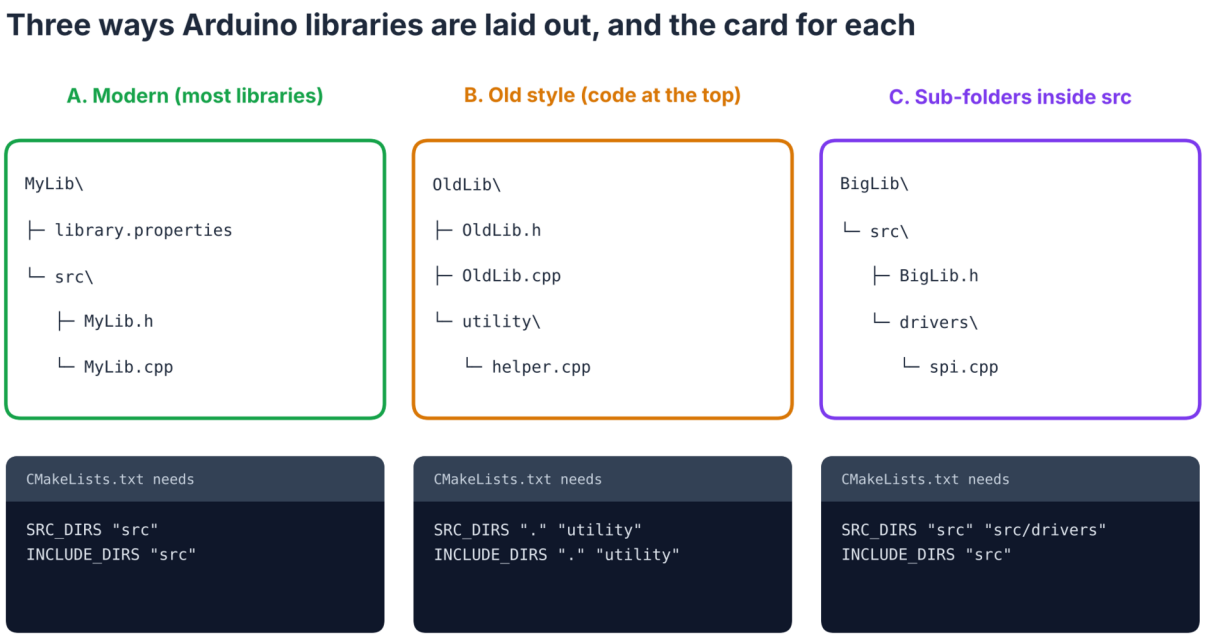
CHAPTER 4

The general recipe for any library

MD_Parola and MD_MAX72XX were well behaved. Here is how to handle the other cases you will meet.

4.1 Library folder layouts

Open the library folder with `dir` before writing its card. Arduino libraries come in three shapes, and the card has to say where the code really is:



SRC_DIRS does not look inside sub-folders by itself, so list each folder that holds .c/.cpp files.

Figure 16. *Look inside the library first, then point SRC_DIRS and INCLUDE_DIRS at the folders that hold the code.*

4.2 Libraries that need other libraries

Many libraries build on others, as MD_Parola builds on MD_MAX72XX. To find out, open the library's `library.properties` and look for a `depends=` line. For each name listed:

1. Copy that library into `components` too.
2. Give it its own `CMakeLists.txt`.
3. Add its folder name to `REQUIRES` in the library that needs it, and in `main` if your sketch includes its header.

For example, Adafruit's display libraries need **Adafruit_GFX**, which in turn needs **Adafruit_BusIO**. All of them go into `components`, each with its own card.

4.3 Never copy built-in libraries

`Wire`, `SPI`, `WiFi`, `Preferences`, `WebServer` and the other libraries that come with the ESP32 Arduino core are already in your project. If your sketch uses them, just keep `arduino-esp32` in `REQUIRES`.

> **COMMON MISTAKE**
> Your Arduino libraries folder may contain a folder called `SPI` or `Wire` that was installed separately. Do **not** copy it into `components`: two libraries with the same name clash, and the build fails with errors such as “multiple definition of …”.

4.4 Libraries that won’t work on an ESP chip

Some libraries were written only for the classic Arduino Uno and similar boards (AVR chips). Check `architectures=` in `library.properties`: `*` means any board, while `avr` alone means AVR only. AVR-only libraries often fail with errors mentioning files such as `avr/io.h` or names such as `PORTB`. Look for an ESP32-compatible alternative instead.

4.5 The checklist for any project

1. List the `#include` lines. Separate built-in libraries from installed ones.
2. For each installed library, check `depends=` and add those libraries to the list.
3. Copy hello_world’s five recipe files into a new folder (no spaces in the path).
4. Copy each installed library into `components\`.
5. Look inside each library and write its `CMakeLists.txt` (SRC_DIRS, INCLUDE_DIRS, REQUIRES).
6. Paste the sketch into `main\main.cpp`, add `#include "Arduino.h"`, and move any pins that aren’t safe.
7. List every installed library your sketch includes in `main\CMakeLists.txt` REQUIRES.
8. `idf.py set-target <chip>`, `idf.py build`, wire up, `idf.py -p COMx flash monitor`.

> **NEW WORDS IN THIS SECTION**
> **library.properties:** the Arduino IDE’s description file for a library: name, version, depends=, architectures=.
>
> **depends=:** the line in library.properties that lists other libraries this one needs.
>
> **architectures=:** the line that says which kinds of chip a library supports (* = all).
>
> **Bit-banging:** switching ordinary pins on and off in software to send data, instead of using a hardware SPI or I2C unit.

CHAPTER 5

Troubleshooting

Find the **first** red `error:` line in the output (later ones are usually knock-on effects), then match it below.

What you see	Likely cause	Fix
`fatal error: X.h: No such file or directory`	The component that includes X.h doesn't have X's folder name in its REQUIRES, or X has no CMakeLists.txt.	Add the folder name to REQUIRES (in main and/or the library). Check X has a card with the right INCLUDE_DIRS.
`Failed to resolve component 'X'`	A REQUIRES name doesn't match any folder.	Use the exact folder name, capitals included (`MD_MAX72XX`).
`undefined reference to ...` (linker)	The header was found, but the .cpp code wasn't compiled.	Point SRC_DIRS at the folder(s) that really contain the .cpp files (Chapter 4.1).
`multiple definition of ...`	The same library is in the project twice (e.g. a copied SPI next to the core's).	Remove the duplicate from components (Chapter 4.3).
Errors mentioning `avr/...` or `PORTB`	AVR-only library.	Find an ESP32-compatible library (Chapter 4.4).
Strange errors after copying a project	The `build` folder was copied too.	`idf.py fullclean`, then build again.
Board keeps restarting after flashing	A sketch uses flash/PSRAM pins (GPIO 14–17, 19–21 on the C61).	Move to free pins (Chapter 2).
Display dark	Wiring, power, or wrong pins in main.cpp.	Recheck the five wires against the table in Step 8; make sure the board is powered.
Random dots, or text mirrored or scrambled	Wrong HARDWARE_TYPE, or 3.3 V signals too weak.	Try `PAROLA_HW` or `GENERIC_HW` instead of `FC16_HW`; try powering the module from 3V3.
`Copy-Item` can't find the path	Documents is (or isn't) inside OneDrive.	Use the path that worked with `dir` in Step 1.

CHAPTER 6

Quick reference

A library's CMakeLists.txt (modern layout)

```
components\<Library>\CMakeLists.txt
idf_component_register(SRC_DIRS "src"
                       INCLUDE_DIRS "src"
                       REQUIRES arduino-esp32 <other libraries it needs>)
```

main's CMakeLists.txt

```
main\CMakeLists.txt
idf_component_register(SRCS "main.cpp"
                       INCLUDE_DIRS "."
                       REQUIRES arduino-esp32 <libraries your sketch includes>)
```

Commands used in this lab

Command	What it does
`dir <folder>`	List what is in a folder.
`type <file>`	Print a text file in the terminal.
`mkdir a\b`	Create folder a and folder b inside it.
`copy <file> <folder>\`	Copy one file (or several, with *).
`Copy-Item -Recurse <folder> <dest>\`	Copy a whole folder with everything inside.
`notepad <file>`	Open (or create) a file in Notepad.
`idf.py set-target esp32c61`	Choose the chip, create sdkconfig, plan the build.
`idf.py build`	Compile and link everything.
`idf.py -p COMx flash monitor`	Build if needed, write to the board, show serial output.
`idf.py fullclean`	Delete all build output for a fresh start.

CHAPTER 7

Glossary

New words from this lab. Words from Lab 1 are in the Lab 1 glossary.

Word	Meaning
Absolute path	A full path including the drive letter, e.g. D:\esp\projects\hello_world.
architectures=	Line in library.properties naming the chips a library supports (* = all).
Bit-banging	Sending data by switching ordinary pins in software instead of using hardware SPI/I2C.
components folder	A folder named components inside a project; ESP-IDF builds each sub-folder that has a CMakeLists.txt.
depends=	Line in library.properties listing other libraries this one needs.
FC16	A common type of 4-in-1 MAX7219 module; selected with FC16_HW.
Frame	One step of an animation; Parola shifts the text one column per frame.
Header file (.h)	A file that declares what a library offers so other code can use it.
INCLUDE_DIRS	CMakeLists keyword: where this component's header files are.
library.properties	The Arduino IDE's description file for a library.
MAX7219	A chip that drives an 8×8 grid of LEDs; four are chained on the display module.
PSRAM	Extra RAM in a separate chip, connected through SPI wires.
Relative path	A path without a drive letter, looked up from the current folder.
REQUIRES	CMakeLists keyword: other components this one needs and whose headers it may include.
SPI bus	A set of wires (clock, data, chip select) for talking to chips such as flash memory.
SRC_DIRS / SRCS	CMakeLists keywords: folders whose code to compile / individual files to compile.

CHAPTER 8

Check your understanding

Answer in your own words first, then check the answers on the next page.

1. Why can’t ESP-IDF use your Arduino libraries folder directly?
2. Your sketch uses Wire.h and Adafruit_SSD1306.h. Which do you copy into components?
3. What does REQUIRES control?
4. Why did the first build fail with “MD_Parola.h: No such file”?
5. Why does MD_Parola’s card need MD_MAX72XX in REQUIRES?
6. Why can’t the display use GPIO 19, 20 and 21 on the C61?
7. Why don’t you copy the build folder into a new project?
8. A library has its .cpp files at the top level and in a utility folder. What goes on its card?
9. You get “undefined reference to MyLib::begin()”. What is the likely cause?
10. Why did the second build (after changing the text) take seconds?

> NOTE
>
> Checkpoint answers from the chapters:
>
> **Chapter 1**: only Adafruit_SSD1306 (and the libraries in its depends= line). Wire is built into the Arduino core.
>
> **Chapter 2**: GPIO 9 is the BOOT strapping pin. If the button holds it low while the board starts, the chip enters download mode instead of running your program.

Answers

1. ESP-IDF only builds components, folders that contain a CMakeLists.txt. Arduino libraries don't come with one, and ESP-IDF doesn't search your libraries folder.
2. Only Adafruit_SSD1306 (plus anything in its depends= line, such as Adafruit_GFX and Adafruit_BusIO). Wire is built into the Arduino core.
3. Which other components a component needs, and therefore whose header files it is allowed to #include.
4. main's CMakeLists.txt (copied from hello_world) didn't list MD_Parola in REQUIRES, so main couldn't see its header.
5. MD_Parola's own code includes MD_MAX72xx.h, so it must be able to see that component's headers.
6. They are the SPI wires the processor uses to read its program from flash memory. Using them corrupts that reading and crashes the board.
7. It remembers the full path of the old project, so it would point to the wrong place. ESP-IDF makes a fresh one.
8. SRC_DIRS "." "utility" and INCLUDE_DIRS "." "utility".
9. The header was found but the library's .cpp files weren't compiled: SRC_DIRS doesn't point at the folder that contains them.
10. It is an incremental build: only main.cpp was recompiled and relinked; the libraries and core were unchanged.

CHAPTER 9

What’s next

You now have the full path from an Arduino sketch to a running ESP-IDF project. Good next steps:

- **Port a second project on your own** with the checklist in Chapter 4.5, ideally one with a sensor library such as a DHT temperature sensor.
- **Explore `idf.py menuconfig`**: the settings menu behind the control ESP-IDF gives you (memory layout, clock, Wi-Fi, FreeRTOS).
- **Share your projects**: a ported project folder (without build, managed_components and sdkconfig) is small enough to put on GitHub.

Versions used in this lab: ESP-IDF v5.5.5, arduino-esp32 3.3.12, MD_Parola and MD_MAX72XX from the Arduino Library Manager. Tested on an ESP32-C61-DevKitC-1, October 2026.

Prepared by Pragyan Sen Deka , Mtech Hybrid Student , Robotics and AI , IITG , Associate IIM Calcutta Innovation Park Technology and Innovation Council This is an independent guide. It is not an official publication of IIT Guwahati or IIMCIP.